Master DSC / MLDM

Minimum Set Cover

Group Description and Provisional Planning

Khayr Eddine LAICHE
Anas BEN ABOU
Rayan LA MONICA
Badis BENAMRAOUI
Mohamed BENGUEZZOU

05/11/2026

1 Project breakdown

We organize the project into three phases: an initialization phase followed by two main work phases, algorithm development and experimental analysis. The dates below are provisional and may be adjusted.

M1 — Initialization and organization. *November 5–8, 2026.* Establish the planning, responsibilities and shared code base. This phase provides a common foundation for the team to start development.

M2 — Algorithm development and validation. *November 9–29, 2026.* Each member develops their assigned algorithms, with mutual support, testing and progressive integration. By the end of this phase, the algorithms must work correctly together and be ready for experimental evaluation.

M3 — Experiments and in-depth analysis. *November 30–December 14, 2026.* Run the experiments and interpret the results through a thorough comparison of the algorithms. This phase produces an analysis of their performance, solution quality, strengths and limitations, supported by experimental evidence.

2 Group members and workload

Member	**Algorithms**	**Other tasks**	**Optional learning**	**Load**
LAICHE Khayr Eddine	Brute force	Shared code base, experimental protocol, statistics	Hardness prediction	20%
BEN ABOU Anas	Dynamic programming (bottom-up and memoized)	Reference solver (HiGHS), shared code base	Labelled training data	20%
LA MONICA Rayan	Greedy and variants	Graphical interface	Learned greedy score	20%
BENAMRAOUI Badis	Branch and bound (two versions)		Learned branching	20%
BENGUEZZOU Mohamed	RWLS, simulated annealing	Real-world application		20%

This distribution is provisional. The second branch-and-bound method and RWLS cover the additional-method and heuristic requirements. Further algorithm variants, learning extensions and the graphical interface will be explored if time permits. The final report will include the actual workload distribution, the provisional and actual planning, and the required completion checklist.

3 Organization and working methodology

We have chosen GitHub to organize our work and manage the project schedule. Our project board provides a Kanban view for task progress and a roadmap for milestone dates. Issues describe tasks and responsibilities; labels categorize the work and identify optional extensions. We will update issue statuses and record actual effort in the *Time spent (h)* field, keeping a history of changes for the final planning review.

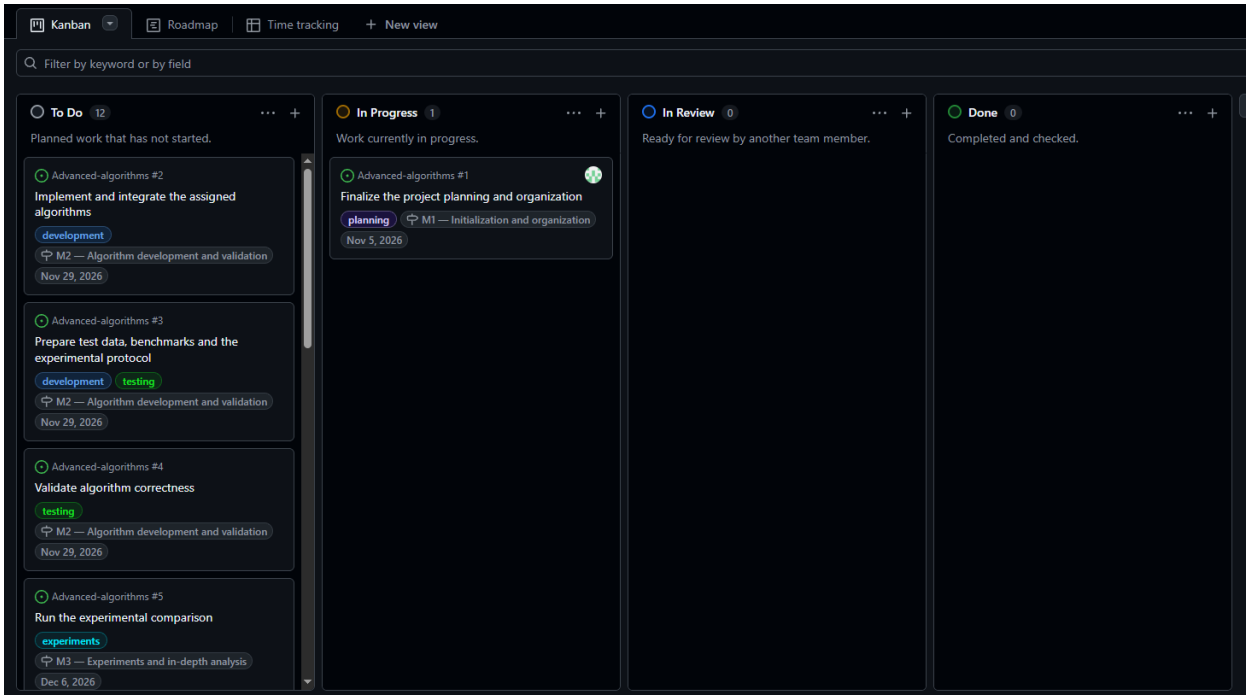


Figure 1: GitHub project: Kanban view.

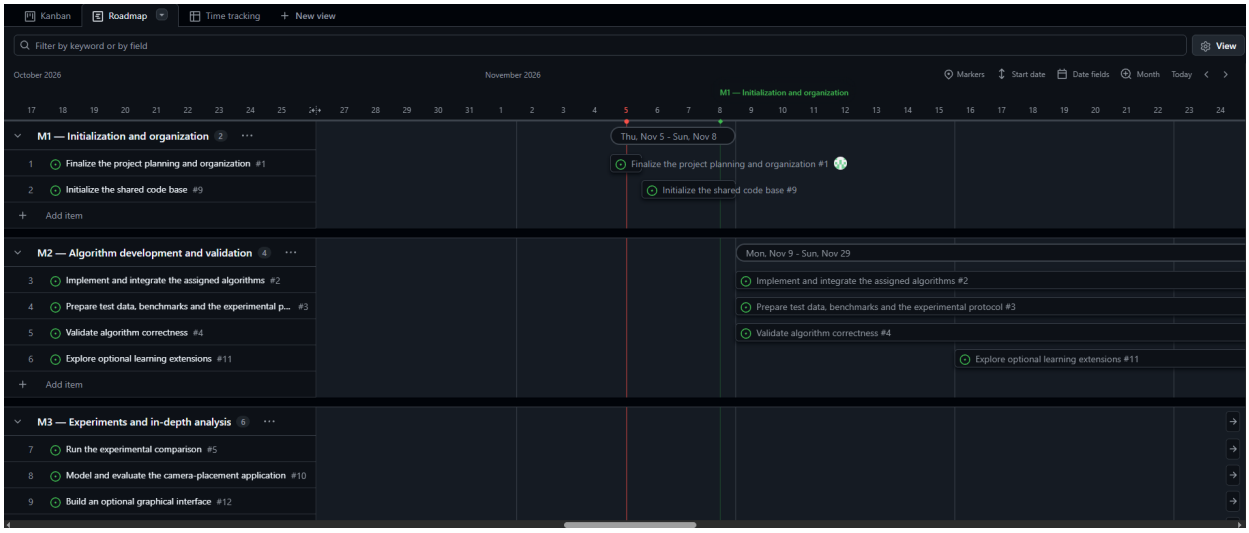


Figure 2: GitHub project: roadmap view.

4 Meetings with the supervisor

The six supervision meetings will review progress within the three project milestones. The table below outlines what we aim to have completed by each meeting; dates and targets will be agreed with the supervisor.

Meeting	Milestone	Expected progress
1	M1	Project organization and responsibilities agreed; shared code base established.
2	M2	First algorithm implementations available, with initial tests on small instances.
3	M2	All required methods implemented and integrated; cross-checks underway and remaining issues identified.
4	M2	Algorithms validated and ready for comparison; benchmark selection and experimental protocol prepared.
5	M3	Main experiments completed; first comparison tables, figures and interpretations available.
6	M3	In-depth comparison and conclusions completed; report draft and presentation outline ready for feedback.

Optional extensions will also be reviewed according to progress and available time. The final archive is due on December 15 before 22:00; the defense takes place on December 17, with slides uploaded just before the assigned slot.

5 Testing and validation

- Every returned cover will be checked automatically: all elements must be covered, and the reported cardinality must match the selected subsets.
- On small feasible instances, exact methods that complete their search must agree. HiGHS will provide a reference optimum only when its solver status confirms optimality.
- A valid cover cannot be smaller than a proven optimum or $\lceil z_{LP} \rceil$, where z_{LP} is the optimum of the linear relaxation. A branch-and-bound lower bound must not exceed the optimum of the corresponding subproblem; it may exceed the global optimum for a branch that can be pruned.
- Interrupted runs will be recorded separately; a feasible solution found before a timeout will not be presented as optimal without a proof.
- Unit tests for each method, fixed random seeds, and every result stored with its parameters so that each experiment can be rerun.

6 Scientific context

Problem. Given a universe U of n elements and m subsets $S_1, \ldots, S_m$ of U, find the smallest number of subsets whose union is U. All subsets have the same cost. The decision version is NP-complete [7]. Note that the OR-Library files use the opposite notation (m rows for the elements, n columns for the subsets).

Question. For a given instance, which method gives the best solution within a fixed time budget, and up to which size can optimality be proved?

- **H1, limits of exact methods.** Brute force trying solution sizes in increasing order examines at most $\sum_{k=0}^{OPT} \binom{m}{k}$ candidate collections on a feasible instance. Dynamic programming over covered elements has up to 2^n states and $O(m \cdot 2^n)$ transitions. We expect dynamic programming to be more competitive on generated instances with few elements and many subsets. STS27 ($m = 27$, $n = 117$) illustrates the different scaling limits; practical runtime and memory limits will be measured.
- **H2, heuristics across families.** We hypothesize that greedy has larger gaps to reference solutions on CLR than on CYC, and that local search reduces these gaps within the time budget. This will be tested experimentally.
- **H3, learning.** We hypothesize that learned components improve the classical methods on instances drawn from the training distribution, with smaller gains on unseen instance types and benchmark families.

Methods. Exact: brute force; dynamic programming (bottom-up and memoized); branch and bound v1, with a feasible greedy solution as upper bound and a counting lower bound (if U' is the set of uncovered elements, at least $\lceil |U'| / \max_j |S_j \cap U'| \rceil$ more subsets are needed, considering only eligible subsets and handling zero gain separately); branch and bound v2, with instance reductions and a Lagrangian lower bound [1]. Approximate: greedy [2] and variants, simulated annealing [9], and RWLS [4]. The integer program solved with HiGHS[1] will be used only as a reference, not counted as one of our methods; its LP solver is described in [6].

7 Instances, evaluation and extensions

Generated instances will be controlled by n, m, the density and a random seed, with every element covered by at least one subset. We will record the realized density and any changes made to ensure coverage. We plan to use unweighted instances from the following families in the USCP_instances repository[2], selecting sizes compatible with the available time and memory:

Family (origin)	Count	Elements	Subsets	Status of the reference value
E	5	50	500	Reference value and optimality status to be established
STS [3]	11	12 to 796 797	9 to 2 187	Reference values and proofs to be checked in [12, 13] and their references
CLR [5]	4	511 to 4 095	210 to 715	Reference values and proof status to be checked in the literature
CYC [5]	6	240 to 28 160	192 to 11 264	Reference values and proof status to be checked in [4, 13]

Reference values will be checked against [10, 13] and any available optimality proofs. Initial HiGHS runs with a 60 s limit per instance are planned to establish reference solutions where possible.

Evaluation. Our implementations will use Python, the same machine and a common time budget; randomized methods will use 10 seeds. We will report runtime, memory use, explored nodes

[1] `https://highs.dev/`

[2] `https://github.com/pinam45/USCP_instances`

where applicable, and gaps to reference values and valid linear-relaxation bounds. Gaps to best-known values will be distinguished from gaps to proven optima. Repeated measurements will include variability; timeouts and memory limits will be reported explicitly.

Optional learning extensions. If time permits, we will explore three components, each compared with the classical method without learning: prediction of the branch-and-bound running time from instance features [11], compared also with a constant predictor; a greedy score learned from solutions whose optimality HiGHS has proved; and a cheap model imitating an expensive branching rule [8]. Training and evaluation instances will be kept separate.

Real-world application and optional interface. We will model surveillance-camera placement as Set Cover [10], using a small simulated case with predefined coverage: points to monitor are the elements, and each candidate camera position covers a known set of points. Cameras have equal cost. We will solve the case with our algorithms and interpret the results. A graphical interface may be added if time permits.

References

[1] J. E. Beasley. An algorithm for set covering problem. *European Journal of Operational Research*, 31(1):85-93, 1987.

[2] V. Chvátal. A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3):233-235, 1979.

[3] D. R. Fulkerson, G. L. Nemhauser, L. E. Trotter. Two computationally difficult set covering problems that arise in computing the 1-width of incidence matrices of Steiner triple systems. *Mathematical Programming Study*, 2:72-81, 1974.

[4] C. Gao, X. Yao, T. Weise, J. Li. An efficient local search heuristic with row weighting for the unicost set covering problem. *European Journal of Operational Research*, 246(3):750-761, 2015.

[5] T. Grossman, A. Wool. Computational experience with approximation algorithms for the set covering problem. *European Journal of Operational Research*, 101(1):81-92, 1997.

[6] Q. Huangfu, J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119-142, 2018.

[7] R. M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85-103. Springer, 1972.

[8] E. B. Khalil, P. Le Bodic, L. Song, G. Nemhauser, B. Dilkina. Learning to branch in mixed integer programming. In *Proceedings of AAAI*, pages 724-731, 2016.

[9] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671-680, 1983.

[10] J. Kritter, M. Brévilliers, J. Lepagnot, L. Idoumghar. On the optimal placement of cameras for surveillance and the underlying set cover problem. *Applied Soft Computing*, 74:133-153, 2019.

[11] K. Leyton-Brown, E. Nudelman, Y. Shoham. Empirical hardness models: methodology and a case study on combinatorial auctions. *Journal of the ACM*, 56(4), 2009.

[12] J. Ostrowski, J. Linderoth, F. Rossi, S. Smriglio. Solving large Steiner triple covering problems. *Operations Research Letters*, 39(2):127-131, 2011.

[13] M. Pinard, L. Moalic, M. Brévilliers, J. Lepagnot, L. Idoumghar. A memetic approach for the unicost set covering problem. In *Learning and Intelligent Optimization (LION 14)*, LNCS, pages 233-248. Springer, 2020. doi:10.1007/978-3-030-53552-0_23.